

LAPORAN PRAKTIKUM MOBILE PROGRAMMING
MODUL 10
(Desain GUI, Navigasi, dan Integrasi API dengan JSON Serialization di
Flutter)



Nama : Dimas Bagus Hari Murti

NIM : 240605110080

Kelas : B

Tanggal : 8 September 2025

JURUSAN TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM
MALANG
GANJIL 2025/2026

I. Tujuan

Tujuan dari percobaan ini untuk memberikan pemahaman mendalam mengenai pengembangan aplikasi mobile berbasis Flutter yang terintegrasi dengan sumber data eksternal. Melalui kegiatan ini, praktikan diharapkan mampu merancang antarmuka pengguna (Graphical User Interface) yang interaktif dan responsif dengan memanfaatkan widget daftar dinamis seperti ListView dan Card. Selain aspek tampilan, praktikum ini juga menargetkan penguasaan sistem navigasi yang terstruktur menggunakan mekanisme named route, yang memungkinkan pengelolaan alur antarhalaman menjadi lebih rapi dan efisien. Fokus utama lainnya adalah kemampuan mengintegrasikan aplikasi dengan API eksternal menggunakan protokol HTTP, serta menerapkan teknik manajemen data modern melalui proses JSON serialization dan deserialization. Dengan demikian, praktikan dapat mengonversi data mentah dari server menjadi objek Dart secara otomatis menggunakan pustaka json_serializable, meminimalkan kesalahan manual, dan menyajikan data tersebut ke layar aplikasi secara real-time .

II. Langkah Kerja

- Membuat proyek Flutter baru bernama named_route_app.
- Membuat beberapa halaman (screens), misalnya HomePage, dan DetailPage.
- Menambahkan konfigurasi named route pada file main.dart menggunakan MaterialApp dengan properti routes.
- Mengimplementasikan navigasi antar halaman menggunakan Navigator.pushNamed() dan Navigator.pop().
- Menambahkan tombol navigasi di setiap halaman untuk berpindah ke halaman lain.
- Menjalankan aplikasi untuk memastikan navigasi berjalan sesuai rute yang telah ditentukan.

III. Screenshot Hasil

1. Main.dart

Source code:

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import '../view/home.dart';
import '../view/detail.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    SystemChrome.setSystemUIOverlayStyle(const SystemUiOverlayStyle(
      statusBarColor: Colors.transparent,
      statusBarIconBrightness: Brightness.dark,
    )); // [cite: 66]

    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Game Store',
      initialRoute: '/', // [cite: 66]
      routes: {
        '/': (context) => const Home(),
        '/detail': (context) {
          final gameId = ModalRoute.of(context)?.settings.arguments as int;
          return Detail(gameTerpilih: gameId);
        },
      }, // [cite: 66]
    ); // [cite: 66]
  }
}
```

File diatas merupakan gerbang utama berjalannya aplikasi. Di sini, saya tidak hanya sekedar me-load tampilan awal, tetapi juga mendefinisikan sistem navigasi aplikasi menggunakan mekanisme Named Route. Dengan mendaftarkan rute '/' untuk Home dan '/detail' untuk Detail, manajemen perpindahan halaman menjadi lebih terpusat. Hal ini memudahkan saya dalam memelihara alur aplikasi dan memastikan parameter (seperti ID game) dapat dikirim antarhalaman dengan cara yang terstruktur dan baku

2. Fetchgame.dart

```
import 'dart:convert';
import '../model/game.dart';
import 'package:http/http.dart' as http;

Future<List<Game>> fetchGames() async {
  final response = await http.get(Uri.parse('https://www.freetogame.com/api/games'));

  if (response.statusCode == 200) {
    final List<dynamic> jsonData = json.decode(response.body);
    return jsonData.map((item) => Game.fromJson(item)).toList();
  } else {
    throw Exception('Gagal mengambil data game');
  }
}

Future<Map<String, dynamic>> fetchDataFromAPI(int gameId) async {
  final response = await http.get(Uri.parse('https://www.freetogame.com/api/game?id=$gameId'));

  if (response.statusCode == 200) {
    final jsonData = json.decode(response.body);
    if (jsonData != null && jsonData is Map<String, dynamic>) {
      return jsonData;
    } else {
      throw Exception('Data dari API tidak sesuai dengan yang diharapkan');
    }
  } else {
    throw Exception('Gagal mengambil data dari API');
  }
}
```

File ini berperan sebagai lapisan logika (logic layer) yang memisahkan urusan pengambilan data dengan urusan tampilan. Di dalamnya, saya membuat fungsi asynchronous seperti `fetchGames()` dan `fetchDataFromAPI()`. Fungsi-fungsi ini menggunakan protokol HTTP untuk mengirim permintaan GET ke server FreeToGame. Karena proses ini bergantung pada koneksi internet dan waktu respon server, saya menggunakan tipe pengembalian `Future` dan kata kunci `await`. Saya juga menambahkan validasi kode status 200 untuk memastikan bahwa data hanya akan diproses jika server memberikan respon sukses, atau melempar exception jika terjadi kegagalan

3. Home.dart

```
import 'package:flutter/material.dart';
import '../model/game.dart';
import '../viewmodel/fetchgame.dart';

class Home extends StatefulWidget {
  const Home({super.key});

  @override
  State<Home> createState() => HomeState();
}

class HomeState extends State<Home> {
  late Future<List<Game>> gameList; // [cite: 97]

  @override
  void initState() {
    super.initState();
    loadGames(); // [cite: 97]
  }

  Future<List<Game>> loadGames() async {
    gameList = fetchGames();
    return gameList;
  } // [cite: 97]

  // Widget Search Bar
  Widget _searchBar() {
    return TextField(
      cursorColor: Colors.blue,
      decoration: InputDecoration(
        fillColor: Colors.blue.shade50,
        filled: true,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(20),
          borderSide: const BorderSide(width: 0, style: BorderStyle.none),
        ),
        contentPadding: const EdgeInsets.symmetric(vertical: 0, horizontal: 20),
        prefixIcon: const Icon(Icons.search_outlined, color: Colors.blue, size: 30),
        hintText: 'Cari game',
        hintStyle: TextStyle(fontSize: 14, color: Colors.grey.withOpacity(0.7)),
      ),
    );
  }

  return Text('Error: ${snapshot.error}');
} else if (snapshot.hasData && snapshot.data!.isEmpty) {
  return const Text('Tidak ada data game.');
```

```

GameListItem(String urlCover, String judul, String genre) {
  return Card(
    clipBehavior: Clip.antiAlias,
    elevation: 5,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
    child: ListTile(
      tileColor: Colors.blue.shade50,
      leading: SizedBox(
        width: 75,
        height: 75,
        child: ClipRect(
          borderRadius: BorderRadius.circular(8),
          child: Image.network(urlCover, scale: 3, fit: BoxFit.cover),
        ),
      ),
      title: Text(judul),
      subtitle: Text(genre),
      trailing: const Icon(Icons.more_vert),
      isThreeLine: false,
      titleAlignment: ListTileTitleAlignment.center,
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.amberAccent.shade50,
    body: SafeArea(
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
        child: Column(
          children: [
            _searchBar(), // [cite: 195]
            const SizedBox(height: 10),
            FutureBuilder<List<Game>>(
              future: fetchGames(),
              builder: (context, snapshot) {
                if (snapshot.connectionState == ConnectionState.waiting) {
                  return const Expanded(child: Center(child: CircularProgressIndicator()));
                } else if (snapshot.hasError) {
                  return Text('Error: ${snapshot.error}');
                }
              },
            ),
          ],
        ),
      ),
    ),
  );
}

```

File ini bertanggung jawab atas antarmuka halaman beranda aplikasi. Saya menggunakan widget FutureBuilder sebagai kerangka utamanya untuk memantau status pengambilan data dari viewmodel. Widget ini memungkinkan saya menampilkan indikator loading saat data sedang diambil, dan menampilkan ListView berisi daftar game saat data berhasil diterima. Selain itu, saya membungkus setiap item dalam daftar tersebut dengan GestureDetector. Logika ini saya pasang agar ketika pengguna menekan salah satu game, aplikasi akan melakukan navigasi ke halaman detail sembari mengirimkan ID game tersebut sebagai argumen .

4. detail.dart

```

import 'package:flutter/material.dart';
import '../viewmodels/fetchgame.dart';
import '../models/game.dart';
import 'package:cached_network_image/cached_network_image.dart';

class Detail extends StatelessWidget {
  final int gameId;
  const Detail({super.key, required this.gameId});
  Future<Detail> fetchData() async {
    final jsonData = await fetchDataFromAPI(gameId);
    return Detail.fromJson(jsonData);
  }

  SizedBox _itemList(String url) {
    return SizedBox(
      width: 200,
      child: ClipRect(
        borderRadius: BorderRadius.circular(15),
        child: Image.network(url, fit: BoxFit.cover),
      ),
    );
  }

  Row _requirements(String title, String value) {
    return Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Expanded(flex: 1, child: Text(title, style: TextStyle(color: Colors.black54, fontWeight: FontWeight.bold))),
        Expanded(flex: 1, child: Text(value, style: TextStyle(color: Colors.black54, fontWeight: FontWeight.bold))),
      ],
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.amberAccent.shade50,
      body: FutureBuilder(
        future: fetchData(),
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
        },
      ),
    );
  }
}

```

```

// Detail Detail
Expanded(
  child: Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 10),
    child: Container(
      padding: const EdgeInsets.all(10),
      width: double.infinity,
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(10),
      ),
      child: SingleChildScrollView(
        child: Column(
          children: [
            _itemList(urlCover),
            _requirements("Minimum System Requirements", "Minimum System Requirements"),
            _requirements("Graphics", "Minimum System Requirements: Graphics"),
            _requirements("Storage", "Minimum System Requirements: Storage"),
          ],
        ),
      ),
    ),
  ),
  // List Screenshots Horizontal
  Container(
    height: 200,
    padding: const EdgeInsets.symmetric(vertical: 10),
    child: ListView(
      scrollDirection: Axis.horizontal,
      children: [
        if (game.screenshots.isNotEmpty) ...[
          _itemList(game.screenshots[0].image),
        ],
      ],
    ),
  ),
)

```

```

201         },
202         if (game.screenshots.length > 1) ...1
203             .titleList(game.screenshots[1].image),
204             const screenSize(width: 10),
205             ),
206         },
207         if (game.screenshots.length > 2)
208             .titleList(game.screenshots[2].image),
209         ),
210     },
211     ), ListView
212 }, Container
213 // Papapipi dengan ReadMore
214 Container(
215     padding: const EdgeInsets.symmetric(horizontal: 5, vertical: 5),
216     child: ReadMoreText(
217         game.description,
218         trimLines: 2,
219         colorClickableText: Colors amberAccent.shade400,
220         trimMode: TrimMode.Line,
221         textStyle: TextStyle(color: Colors.black86, width: 0.8, height: 1.5),
222         trimExpandedText: 'more',
223         trimExpandedText: 'less',
224         ), ReadMoreText
225 ), Container
226 ),
227 ),
228 ), SingleChildScrollView
229 ), Container
230 ), Container
231 ), Expanded
232 ),
233 ), Column
234 ),
235 },
236 ),
237 ), FutureBuilder
238 ), Scaffold
239 ),
240 ),

```

5. game.dart

```
class Game {
    final int id;
    final String title;
    final String thumbnail;
    final String shortDescription;
    final String genre;
    final String platform;
    final String publisher;
    final String developer;
    final String releaseDate;
    final String freeToGameProfileUrl;

    Game(
        requires this.id,
        requires this.title,
        requires this.thumbnail,
        requires this.shortDescription,
        requires this.genre,
        requires this.platform,
        requires this.publisher,
        requires this.developer,
        requires this.releaseDate,
        requires this.freeToGameProfileUrl,
    );

    factory Game.fromJson(Map<String, dynamic> json) {
        return Game(
            id: json['id'],
            title: json['title'],
            thumbnail: json['thumbnail'],
            shortDescription: json['short_description'],
            genre: json['game_genre'],
            publisher: json['publisher'],
            platform: json['platform'],
            developer: json['developer'],
            releaseDate: json['release_date'],
            freeToGameProfileUrl: json['free_to_game_profile_url'],
        );
    }
}
```

File ini untuk mendefinisikan struktur data dasar dari sebuah game yang akan ditampilkan pada daftar halaman utama. Kelas Game di dalamnya bertindak sebagai cetakan objek yang menampung atribut-atribut penting seperti judul, thumbnail, dan genre. Saya membuat factory constructor bernama `Game.fromJson` di sini, yang logika utamanya adalah memetakan data mentah berformat JSON dari API menjadi objek Dart. Tujuannya agar data tersebut lebih mudah diolah dan ditampilkan pada widget UI tanpa harus memanggil key string JSON berulang kali .

6. detailgame.dart

```
import 'package:json_annotation/json_annotation.dart';

part 'detailgame.g.dart';

@JsonSerializable()
class MinimumSystemRequirements {
  final String? os;
  final String? processor;
  final String? memory;
  final String? graphics;
  final String? storage;

  MinimumSystemRequirements({
    this.os,
    this.processor,
    this.memory,
    this.graphics,
    this.storage,
  });

  factory MinimumSystemRequirements.fromJson(Map<String, dynamic> json) =>
    _MinimumSystemRequirementsFromJson(json);
  Map<String, dynamic> toJson() => _MinimumSystemRequirementsToJson(this);
}

@JsonSerializable()
class Screenshot {
  final int id;
  final String image;

  Screenshot({required this.id, required this.image});

  factory Screenshot.fromJson(Map<String, dynamic> json) =>
    _ScreenshotFromJson(json);
  Map<String, dynamic> toJson() => _ScreenshotToJson(this);
}
```

file ini menangani struktur data yang jauh lebih kompleks dan bersarang (nested), seperti adanya objek spesifikasi sistem (Minimum System Requirements) dan daftar tangkapan layar (Screenshots) di dalam satu objek detail game. Oleh karena itu, saya memanfaatkan anotasi `@JsonSerializable` pada kelas ini. Dengan anotasi tersebut, proses konversi JSON yang rumit akan ditangani sepenuhnya oleh mesin code generator, memastikan bahwa struktur data yang diterima dari API dapat diproses menjadi objek Dart secara presisi dan efisien .

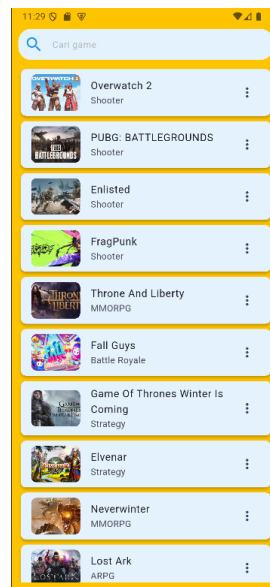
```

39 @JsonSerializable()
40 class DetailGame {
41     final int id;
42     final String title;
43     final String thumbnail;
44     final String status;
45     final String short_description;
46     final String description;
47     final String game_url;
48     final String genre;
49     final String platform;
50     final String publisher;
51     final String developer;
52     final String release_date;
53     final String freetogame_profile_url;
54     final MinimumSystemRequirements minimum_system_requirements;
55     final List<Screenshot> screenshots;
56
57     DetailGame({
58         required this.id,
59         required this.title,
60         required this.thumbnail,
61         required this.status,
62         required this.short_description,
63         required this.description,
64         required this.game_url,
65         required this.genre,
66         required this.platform,
67         required this.publisher,
68         required this.developer,
69         required this.release_date,
70         required this.freetogame_profile_url,
71         required this.minimum_system_requirements,
72         required this.screenshots,
73     });
74
75     factory DetailGame.fromJson(Map<String, dynamic> json) =>
76         _DetailGameFromJson(json);
77     Map<String, dynamic> toJson() => _DetailGameToJson(this);
78 }

```

Output:

Homepage:



Detail game page:



IV. Kesimpulan

Berdasarkan serangkaian percobaan yang telah dilakukan, dapat disimpulkan bahwa integrasi API dan manajemen navigasi yang baik merupakan fondasi utama dalam membangun aplikasi mobile yang berskala dinamis. Penggunaan widget FutureBuilder terbukti sangat efektif dalam menangani proses pengambilan data secara asinkron (asynchronous), memungkinkan aplikasi untuk memberikan umpan balik visual yang tepat kepada pengguna, baik saat proses memuat data (loading), saat terjadi kesalahan (error), maupun ketika data berhasil ditampilkan. Di sisi lain, penerapan named route memberikan solusi navigasi yang lebih terorganisir dibandingkan metode navigasi konvensional, terutama dalam hal skalabilitas dan

keterbacaan kode saat aplikasi memiliki banyak halaman. Selain itu, pemanfaatan teknik kode generatif melalui `json_serializable` dan `build_runner` secara signifikan meningkatkan efisiensi pengembangan dengan mengotomatisasi konversi format JSON ke model objek Dart, sehingga struktur data aplikasi menjadi lebih kuat, konsisten, dan mudah untuk dipelihara .